

M V V M

Model–View–ViewModel Pattern

Bind view to a view-model

DEFINITION

Separate view from logic via a ViewModel that exposes bindable state and commands; the view binds (often two-way) to it.

MVVM replaces hand-written view updates with a binding layer. The ViewModel exposes observable state and commands and holds no reference to the view; the framework keeps the two in sync automatically. In Angular your component is the ViewModel and signals are its bindable state — you write state and intent, the framework renders.

WHY IT MATTERS

When the framework already syncs state to the DOM — Angular signals, Vue refs, WPF bindings — a presenter that manually pushes each update is wasted, error-prone code. MVVM leans entirely on binding: you declare bindable state and commands, and the engine handles the traffic between View and ViewModel in both directions.

— How it works

Model

Domain and data — unchanged from MVC and MVP.

View

A declarative template bound to the view-model. No imperative DOM writes; bindings read state and route events to commands.

ViewModel

Exposes bindable state (signals/observables) and commands. Holds zero reference to the view — that's what keeps it unit-testable as a plain object.

HOW TO APPLY

1. Model UI state as signals/observables on the component (the ViewModel).
2. Bind the template to that state; never write to the DOM by hand.
3. Expose user intent as commands (methods) the template calls.
4. Keep the view-model free of view/DOM references so you can test it directly.

LITMUS TEST

Does your view-model reference the view, the DOM, or ElementRef? If yes, it's a presenter, not a view-model. A view-model exposes only bindable state and commands.

— Smell → fix

A component that reaches into the DOM to push updates by hand:

```
class CounterComponent {  
  count = 0  
  inc() {  
    this.count++  
    this.el.nativeElement.textContent = this.count // manual sync  
  }  
} // view logic tied to the DOM
```

↓ BIND INSTEAD OF PUSH

```
@Component({ template: '{{ count() }}' })  
class CounterComponent {  
  count = signal(0) // bindable state  
  inc = () => this.count.update(n => n + 1) // command  
}  
// template binds to count(); no manual DOM writes
```

The component is now a pure ViewModel: state is a signal, the command updates it, the template binds to `count()`. It never touches the DOM, so you can unit-test it as a plain object. Two-way (`ngModel` / `model()`) closes the loop from view back to view-model.

USE WHEN

- ✓ Data-binding frameworks (Angular, Vue, WPF)
- ✓ Reactive UIs

AVOID WHEN

- ▲ Static pages where binding overhead isn't justified

— Trade-offs & pitfalls

PROS

- + Testable view-model
- + Less glue code via binding

CONS

- Binding "magic" and debugging
- Logic can leak into the view

COMMON PITFALLS

- ▲ Logic leaking into templates — heavy expressions or side effects in bindings.
- ▲ Binding "magic" hiding change-detection bugs that are hard to trace.
- ▲ A view-model that grabs ElementRef or the DOM — it has quietly become a presenter.

WHERE YOU SEE IT

- ▶ Angular (signals + templates), Vue (refs + v-model), WPF/Silverlight (the origin).
- ▶ Reactive forms where state, validation and view stay bound together.
- ▶ Knockout.js — early browser MVVM that popularised the term on the web.

SEE ALSO

MVC	MVVM is the binding-era descendant of MVC/MVP for reactive UIs
Observer	binding is Observer under the hood — the view subscribes to view-model state
MVP	MVVM drops the presenter-to-view calls; the binding layer does the syncing
SoC	state and commands vs rendering — concerns split along the binding boundary

— Interview & sources

Why does MVVM fit Angular and Vue so well?

Their reactive primitives (signals/refs plus templates) provide the ViewModel-to-View sync the pattern needs; two-way directives (ngModel, v-model) close the loop back without glue code.

What must a ViewModel never do?

Reference the view or the DOM. It exposes bindable state and commands only; the binding layer connects them. A view reference turns it into a presenter.

MVVM vs MVP — when would you pick each?

Pick MVVM when the framework gives you data binding (Angular, Vue, WPF) — it removes the presenter's manual sync code. Pick MVP when there's no binding layer and you need testability by hand (classic Android, WinForms).

Where does MVVM go wrong in practice?

Logic leaks into templates (heavy expressions, side effects) and binding "magic" makes change-detection bugs hard to trace. Keep templates declarative and logic in the view-model.

REMEMBER

The view-model never names the view. Bindings carry state out and events back.

SOURCES

John Gossman — MVVM, introduced for WPF/Silverlight (2005)

Martin Fowler — "Presentation Model" (the pattern MVVM specialises)

Angular signals & Vue reactivity — MVVM in modern front-ends